

Digital Egypt Pioneers – Round#4

IBM Data Science Project

Rossmann Store Sales Forecasting and Demand Prediction

• Team Members:

Member Name	Email
Asmaa Elsayed Mohamed Morsy	asmaamorsi555@gmail.com
Aya Ahmed Kamal Abdulhameed	aya98ahmed81@gmail.com
Marvel Baheeg Moneer Azmy	marvelbaheeg@gmail.com
Yasmeen Esmail Mohamed Abdullah	yasmeenesmael23@gmail.com

NHA Group Code: **ONL4_AIS4_G1**

Project Planning & Management

1. Project Proposal

1.1. Project Overview

In the modern retail landscape, accurate sales forecasting is a critical pillar for strategic decision-making. Rossmann, one of Europe's largest drugstore chains operating over 3,000 stores, currently relies on individual store managers to predict daily sales up to six weeks in advance. This decentralized approach leads to significant variance in accuracy due to subjective human judgment and the complexity of overlapping variables.

This project aims to develop a robust Time Series Forecasting model using advanced Machine Learning techniques. By leveraging historical data from 1,115 German stores, the system will automate the prediction process, accounting for complex patterns like seasonality, promotional effects, and local competition to provide high-precision sales estimates.

1.2. Problem Definition

The manual forecasting process at Rossmann faces several challenges:

- Inconsistency: Forecast accuracy varies significantly between different store managers.
- Complexity: Human planners struggle to quantify the simultaneous impact of promotions, school holidays, and competitor openings.
- Operational Risk: Inaccurate forecasts lead to inefficient staff scheduling, resulting in either unnecessary labor costs or understaffed shifts that decrease customer satisfaction and employee motivation.

1.3. Project Objectives

The primary goal of this graduation project is to build an end-to-end data science pipeline that achieves the following:

- Automated Forecasting: Develop a model to predict daily sales for a 6-week horizon across 1,115 stores.
- Accuracy Optimization: Minimize the Root Mean Square Percentage Error (RMSPE) to ensure the model outperforms traditional baseline methods.

- Feature Impact Analysis: Identify and rank the key drivers of sales (e.g., how much a "Promo" increases volume compared to a "State Holiday").
- Decision Support: Provide a reliable data foundation for store managers to optimize resource allocation and inventory planning.

1.4. Project Scope

The scope of this research is divided into four main technical phases:

A. Data Integration & Preprocessing

- Merging multiple datasets (Store features + Historical sales).
- Handling missing values (e.g., competition distance) and cleaning noise.
- Filtering non-operational days (days when stores were closed).

B. Feature Engineering (The Core)

- Temporal Features: Extracting day of the week, month, year, and fiscal quarters.
- Trend Analysis: Implementing "Lag features" and "Rolling averages" to capture momentum.
- External Factors: Encoding categorical variables such as StoreType, Assortment, and holiday flags.

C. Modeling & Algorithms

- Implementing baseline models (Moving Average, ARIMA).
- Developing advanced Regressors such as XGBoost or Random Forest.

2. Project Plan

2.1. Project Timeline

Phase	Activities	Timeline
1. Data Cleaning & Exploring	Handling missing values, duplicates, outliers, and data normalization.	Feb 1 – Feb 14
2. Advanced Data Analysis	Statistical analysis, feature engineering, and data visualization.	Feb 15 – Feb 28
3. Model Building & Deployment	Model selection, training, evaluation, and hyperparameter tuning.	Mar 1 – Mar 14
4. MLOps Implementation	Model deployment, monitoring, and retraining strategies.	Mar 15 – Mar 28
5. BI Dashboard	Creating a dashboard for historical data and sales predictions.	Mar 29 – Apr 11
6. Documentation & Final Delivery	Final report, presentation, and future improvements.	Apr 12 – Apr 25

2.2. Project Resource Allocation

Phase	Team Members	Duration
Data Cleaning	Asmaa Elsayed, Marvel Bahig, Aya Ahmed, Yasmin Ismail	2 Weeks
Advanced Analysis	Asmaa Elsayed, Marvel Bahig, Aya Ahmed, Yasmin Ismail	2 Weeks
Model Building	Marvel Bahig	2 Weeks
MLOps	Asmaa Elsayed	1 Week
Deployment	Asmaa Elsayed, Yasmin Ismail	1 Week
BI Dashboard	Aya Ahmed	2 Weeks
Final Delivery	Asmaa Elsayed, Marvel Bahig, Aya Ahmed, Yasmin Ismail	2 Weeks

3. Risk Assessment& Mitigation Plan

#	Risk Identified	Mitigation Solution
1	High Missing Data Volume: Significant null values in store.csv (e.g., Competition distance and Promo2 dates).	Logical Imputation: Use the Median for CompetitionDistance to avoid outlier bias, and fill date-related nulls with 0 to represent the absence of a competitor or promotion.
2	Feature Mismatch (Customers): The Customers column is available in the training set but missing in the test set.	Feature Exclusion: Exclude the Customers column from the final training features to ensure the model's schema matches the test set requirements while using it only for EDA insights.
3	Extreme Sales Outliers: Massive sales peaks during Christmas and major Promo periods can skew simple models.	Robust Algorithms: Use Tree-based models (like XGBoost or Random Forest) and perform Feature Engineering to add specific holiday flags and date-based features to handle seasonality.
4	Temporal Data Leakage: Traditional random splitting might use future data to predict the past in a time-series context.	Time-Series Split: Implement a chronological validation strategy (Time-Series Split) where the model is trained on past data and validated on subsequent future periods.
5	Censored Demand (Sales Proxy): Recorded sales do not account for "lost orders" during stock-out periods.	Proxy Documentation: Acknowledge "Sales as a Proxy for Demand" as a project limitation and focus on analyzing high-traffic days to recommend inventory safety buffers.
6	Data Type Inconsistency: Inconsistent entries in the StateHoliday column (mixing numeric 0 with string "0").	Standardization: Perform explicit Data Type Casting to convert the entire column into a string format during the preprocessing stage to ensure uniform categorization.

4. Key Performance Indicators (KPIs):

To measure “Rossmann Store Sales” project success, the following KPIs will be tracked:

- **Prediction Accuracy:** Achieve the lowest possible Root Mean Square Error (RMSE) for daily sales forecasts across 1115 stores.
- **Forecast Error Rate:** Minimize the percentage deviation between predicted and actual sales, targeting under 10% for promotional and non-promotional periods.
- **Model Response Time:** Speed of generating sales predictions per store; acceptable latency for real-time or batch use. Ensure rapid model training cycles and real-time prediction delivery under 1 second per store query.
- **Feature Importance Stability:** Consistency of top predictors (promotions, holidays, store type, Customers, Sales Per Customer, ...etc.) across training runs, ensuring the model is learning meaningful patterns.
- **Model Deployment Uptime:** Maintain 99.9% availability of the sales forecasting system with zero unplanned downtimes during peak retail seasons.
- **Business Impact & Operational Excellence:** Transition from the currently "varied" manual predictions made by individual managers to a robust forecasting model that enables effective staff scheduling for 6 weeks. Success is measured by the model's ability to increase productivity and motivation across the 1,115 stores, allowing managers to redirect their focus away from administrative forecasting and toward their customers and teams.

Requirements Gathering

5. Stakeholder Analysis

Identifying and managing stakeholders is critical for this project because the outputs ranging from Sales Forecasts to KPI Dashboards directly influence strategic investments, inventory logistics, and daily store operations.

5.1. Stakeholder Identification & Interests

Stakeholder	Key Interests & Objectives	Role in the Project
Executive Leadership (Executives)	Return on Investment (ROI), waste reduction, and market share growth.	Decision Makers: Rely on the dashboard to make strategic expansion decisions or adjust promotional policies.
Operations & Inventory Managers	Forecast accuracy to prevent out-of-stock scenarios or excessive overstocking.	Primary Model Users: Rely on forecasting outputs to schedule supply chain logistics and inventory replenishment.
IT & DevOps Team	System stability, data security, and seamless MLOps integration with existing infrastructure.	Technical Enablers: Ensure model continuity and successful deployment without impacting point-of-sale (POS) systems.
Store Managers	Easy-to-understand daily forecasts for efficient staff scheduling based on predicted customer traffic.	Field Beneficiaries: Provide critical feedback regarding the real-world accuracy of predictions at the ground level.
Suppliers	Transparency in future demand to ensure flexible and resilient supply chains.	Impacted External Parties: Leverage forecast accuracy to reduce emergency shipping costs and optimize production.

5.2. Power/Interest Matrix

We categorize these stakeholders to prioritize engagement and resource allocation:

High Power / High Interest (Manage Closely):

- Executive Leadership & Operations



- stakeholders with both high power and high interest. These are key players who are heavily invested in the project and have significant influence over it. Project managers should engage with them closely and frequently, involving them in decision-making to ensure their continued support and alignment with project goals.
- These stakeholders define project success. They require deep engagement through the Power BI Dashboard to monitor KPIs and business impact.

High Power / Low Interest (Keep Satisfied):

- IT & Cybersecurity Teams represent stakeholders with high power but low interest. These stakeholders have the ability to influence the project but are not particularly engaged with its details. The strategy is to keep them satisfied with the project's overall progress and outcomes, without overwhelming them with information, to prevent them from using their power to disrupt the project.
- They control the production environment. We must ensure that our MLOps architecture complies with all corporate security and performance standards.

Low Power / High Interest (Keep Informed):

- Store Managers, stakeholders with high interest but low power. These individuals or groups are keen on the project's progress and outcomes but have limited ability to influence it. The recommended strategy is to keep them regularly updated to ensure their support and prevent potential opposition arising from misinformation.
- Their "buy-in" is essential for field adoption. We must provide them with clear, simplified visualizations of the forecasts to assist in their daily routines.

Low Power / Low Interest (Monitor):

- External Suppliers, stakeholders with both low power and low interest. These stakeholders are unlikely to be heavily involved or have a significant impact on the project. The appropriate strategy is to monitor them with minimal effort, as their positions could potentially change over the course of the project.

- They are informed of high-level shifts in demand patterns that may affect long-term procurement.

5.3. Stakeholder Risk Mitigation

Trust Gap: Store managers may prefer "intuition" over model predictions.

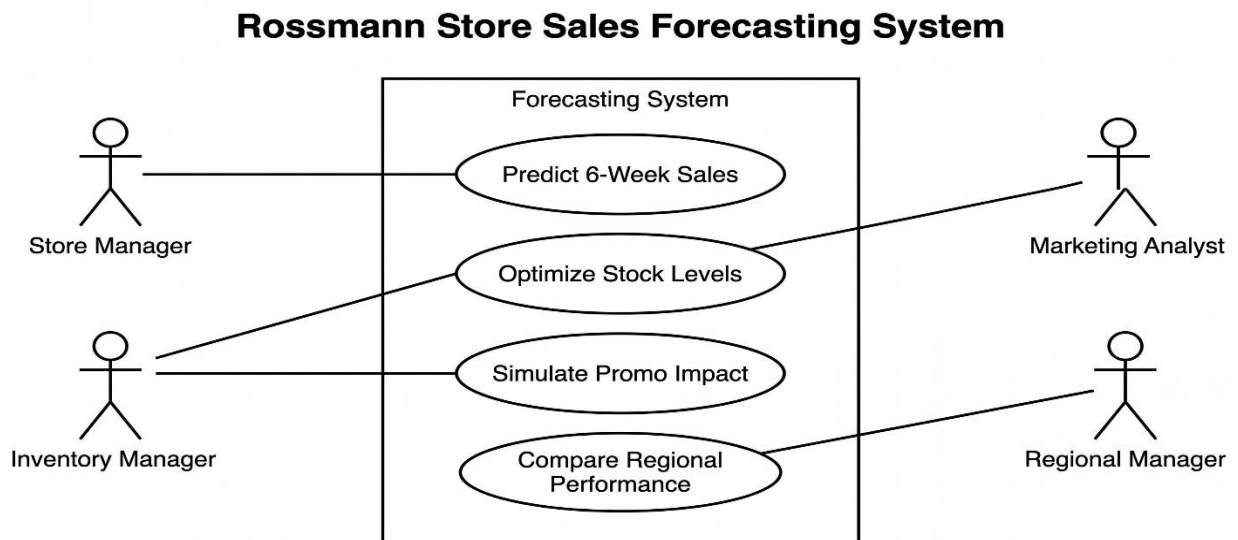
- Mitigation: Use Explainable AI (XAI) techniques to show which features (like Promos or Holidays) are driving a specific forecast.

Expectation Misalignment: Management may perceive a 5% error margin as a "failure."

- Mitigation: Use the Baseline Comparison in Power BI to show how much more accurate the ML model is compared to traditional moving averages.

6. User Stories & Use Cases

6.1. Use Case Diagram



6.2. User Stories

User Role	User Story (As a... I want to... so that...)
Store Manager	As a store manager, I want to see the sales forecast for the next 6 weeks, so that I can manage staff schedules and shifts efficiently.
Inventory Manager	As an inventory manager, I want to predict demand during promotion periods, so that I can ensure stock availability and avoid lost orders.
Marketing Team	As a marketing analyst, I want to simulate the impact of different promotion schedules on predicted sales, so that I can design campaigns that maximize ROI.
Regional Manager	As a regional manager, I want to compare forecasted sales across different store types, so that I can allocate resources and budgets effectively.

6.3. Use Case Scenarios

Scenario A: Operational Planning (Store Manager)

- Actor: Store Manager.
- Goal: Optimize workforce scheduling.
- Flow: The manager opens the dashboard, selects "Staff Planner," and views the predicted sales peaks for the next 14 days. Based on the spikes, the

manager assigns extra staff for Mondays and Promo days.

Scenario B: Inventory Optimization (Inventory Manager)

- Actor: Inventory Manager.
- Goal: Prevent Stock-outs (Out of stock).
- Flow: The system flags stores where a "Promo2" is starting next week. The manager sees a predicted 30% increase in sales and immediately triggers an automated stock replenishment order to the central warehouse.

Scenario C: Campaign Simulation (Marketing Team)

- Actor: Marketing Analyst.
- Goal: "What-If" Analysis for new promotions.
- Flow: The analyst inputs a hypothetical 10-day promotion into the simulator. The model calculates the "Sales Lift" (expected increase). The analyst compares this against the cost of the promo to decide whether to launch it or modify the dates.

Scenario D: Strategic Resource Allocation (Regional Manager)

- Actor: Regional Manager.
- Goal: Quarterly Budget Planning.
- Flow: The manager generates a report comparing "Store Type A" (Urban) vs "Store Type D" (Rural). The model shows higher growth potential in Type D. The manager decides to allocate 20% more marketing budget to Rural stores for the next quarter.

7. Non-functional Requirements

7.1. Performance

The system must be able to generate daily sales forecasts for all 1,115 stores over six weeks. Training the model requires standard hardware, with a forecast response time of less than 100 milliseconds per daily store query. Database queries for features such as promotions or store types must be completed in under 200 milliseconds, with

processing power to support 100 concurrent users during peak analysis periods.

7.2. Security

- **Data Integrity:** The forecasting system must ensure that historical sales data and supplemental store information are protected from unauthorized modifications to maintain the "robustness" of the prediction model.
- **Access Control:** Access to sales forecasts and sensitive data fields like Sales and Customers should be restricted to authorized store managers and operations teams to protect proprietary business insights.

7.3. Usability

The interface should load dashboards in less than 3 seconds, with intuitive visualizations accessible via drag and drop filters for non-technical users like store managers

7.4. Reliability

Achieve 99.9% uptime for core forecasting services, with automated backups of models and data every 24 hours and a recovery point objective (RPO) less than 1 hour. The system must handle failures gracefully

8. functional Requirements

8.1. Data Integration & Management

The system must handle various internal and external data streams to ensure the model has a holistic view of the retail environment.

- **Historical Sales Ingestion:** Automated daily or weekly uploads of historical sales data, including customer counts and turnover.
- **Store Metadata Management:** Ability to categorize stores by type (e.g., a, b, c, d), assortment levels, and geographic competition distance.
- **External Variable Integration:** Automatic synchronization of public holiday calendars and state-specific school holidays.
- **Promotion (Promo) Tracking:** Ability to log and differentiate between standard promotions and "Promo2" (sustained, consecutive promotional cycles).

8.2. Predictive Analytics & Modeling

This is the core engine of the application, responsible for generating the actual forecasts.

- Multi-Horizon Forecasting: Ability to generate short-term (7-day), medium-term (6-week), and long-term (quarterly) sales predictions.
- Automatic Feature Engineering: The system should automatically extract time-based features such as:
 - Day of the week (Monday vs. Sunday trends).
 - Seasonality (Christmas, Easter, and back-to-school surges).
 - Competition impact (how long a nearby competitor has been open).
- Closed-Day Handling: Logic to automatically predict zero sales for days stores are closed (refining the RMSPE accuracy).
- Anomaly Detection: Identifying and flagging "outlier" days (e.g., a sudden local event that spiked sales) to prevent them from skewing future models.

8.3. Visualization & Reporting

Data is only useful if store managers can interpret it quickly.

- Interactive Dashboards: Visual comparisons of "Actual vs. Predicted" sales to build trust in the model.
- Heatmap Analytics: Visualizing sales density across different days of the week or months.
- Store Comparison Tool: Ability for regional managers to compare the performance of multiple stores side-by-side.
- Export Functionality: Exporting forecasts into CSV or PDF formats for integration into existing inventory management systems.

8.4. Scenario Analysis ("What-If" Planning)

This allows users to test strategies before implementing them in the real world.

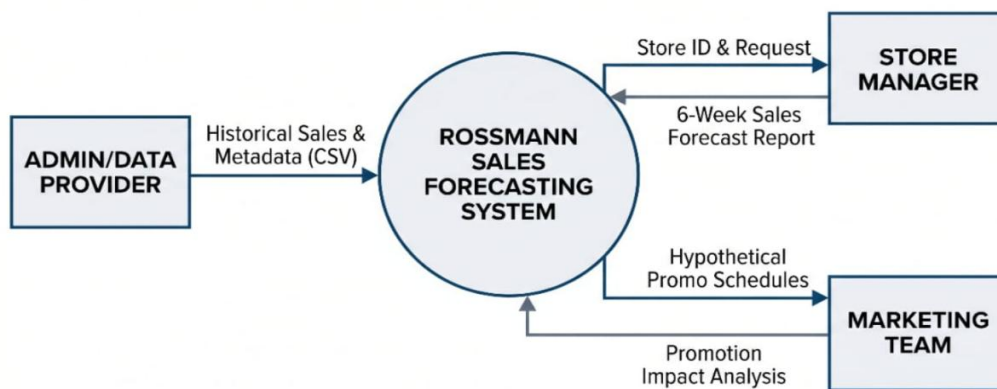
- Promotion Simulation: "What happens to my sales if I run a Promo2 campaign during a school holiday?"
- Competition Impact Modeling: Adjusting the "Competition Distance" variable to see how a new rival store opening nearby might affect revenue.

System Analysis & Design

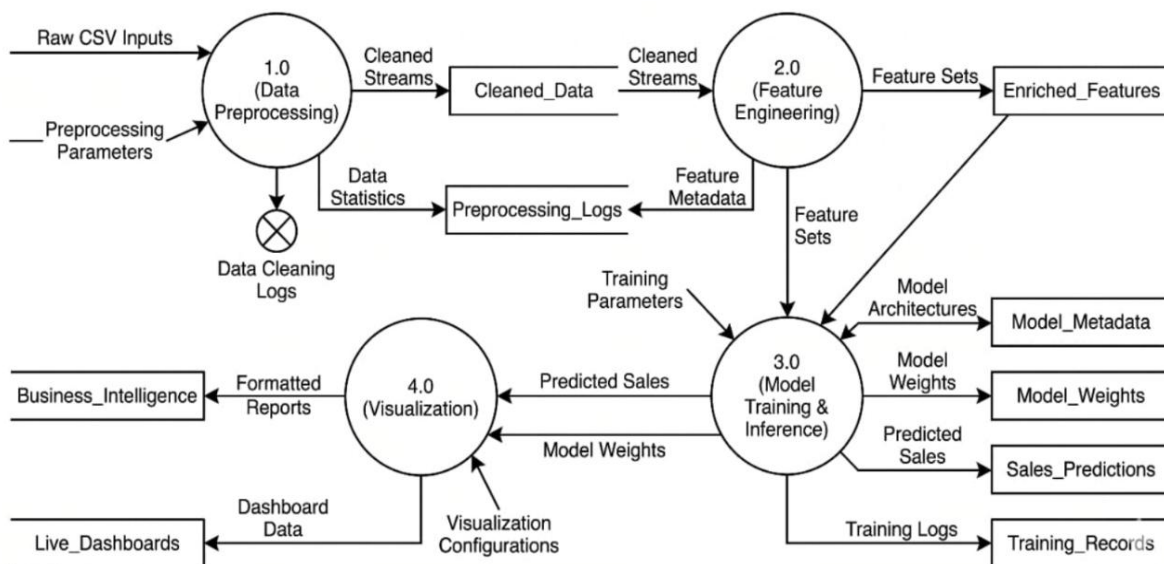
9. Data Flow & System Behavior

9.1. DFD (Data Flow Diagram)

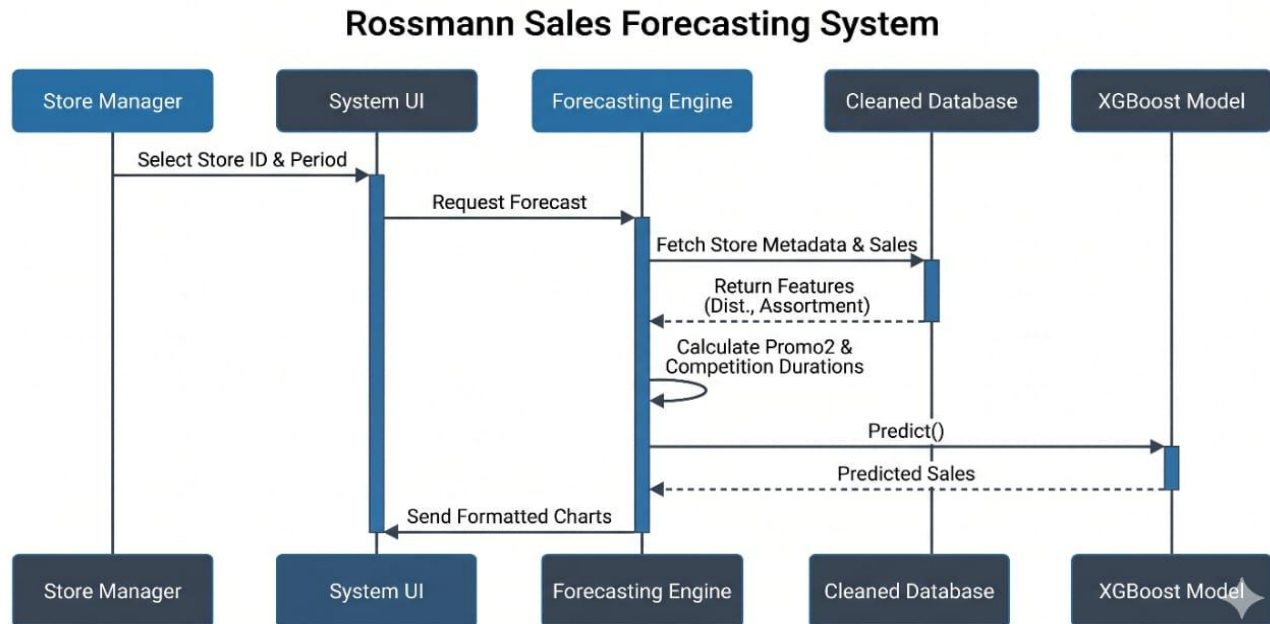
DFD Level 0 (Context Diagram)
'Rossmann Sales Forecasting System'



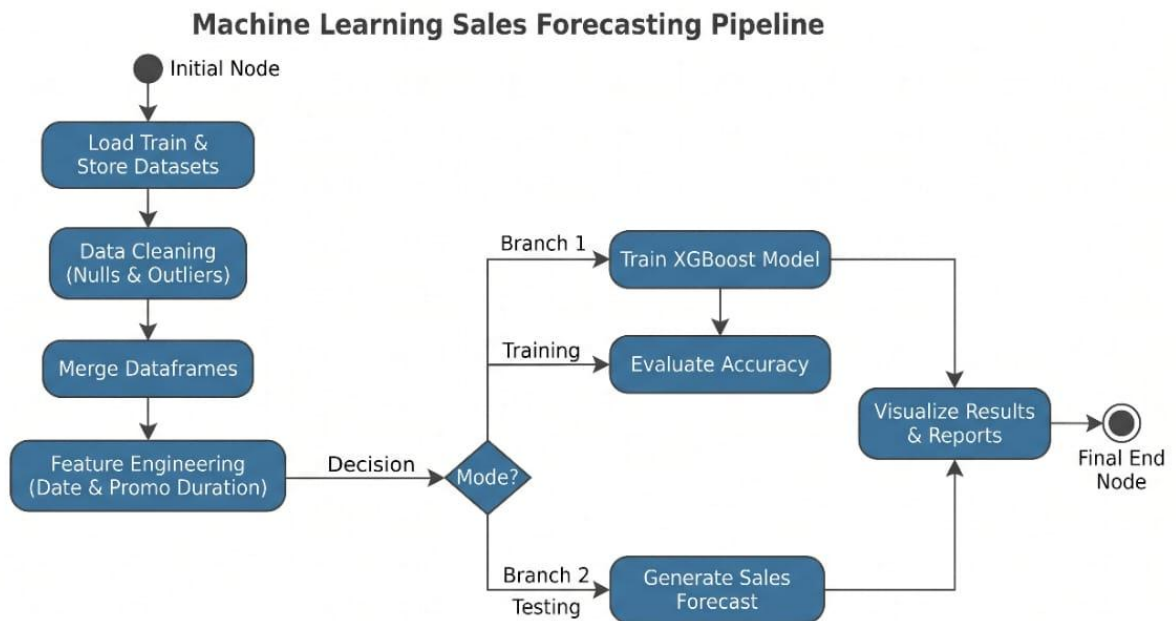
Data Flow Diagram (DFD) Level 1



9.2. Sequence Diagram

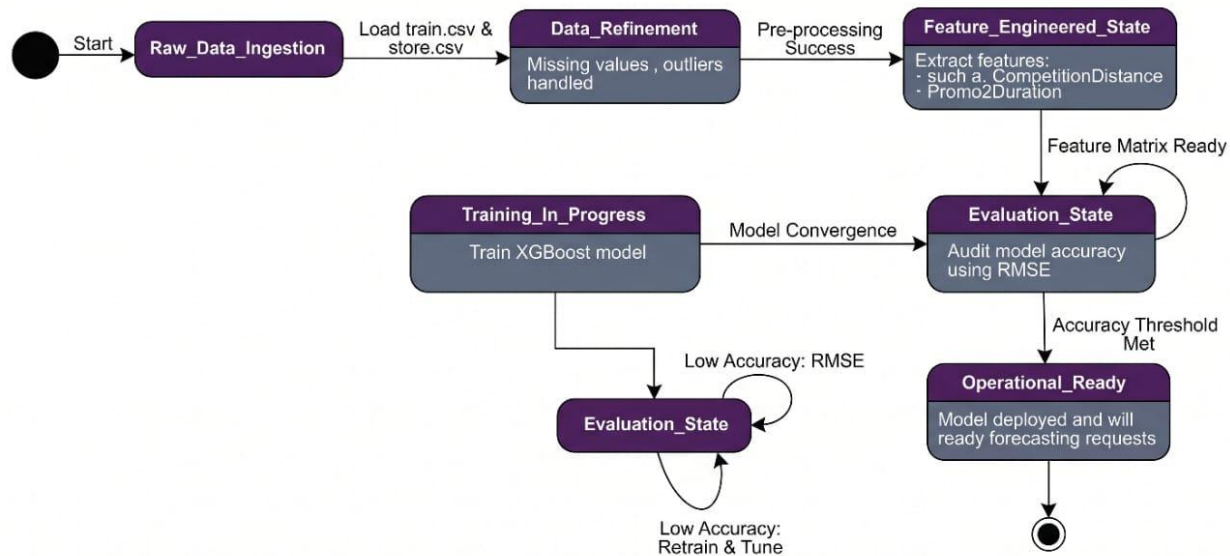


9.3. Activity Diagram



9.4. State Diagram

Machine Learning Sales Forecasting Pipeline



10. UI/UX Design For Dashboard





Open: 0 1

Promo2: 0 1

Promo: 0 1

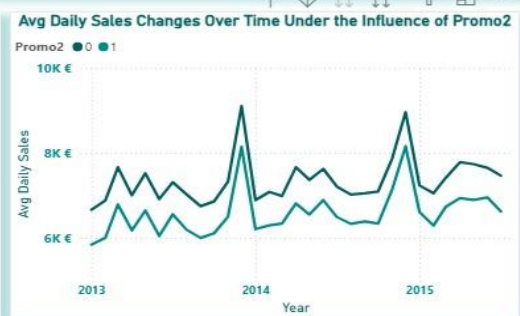
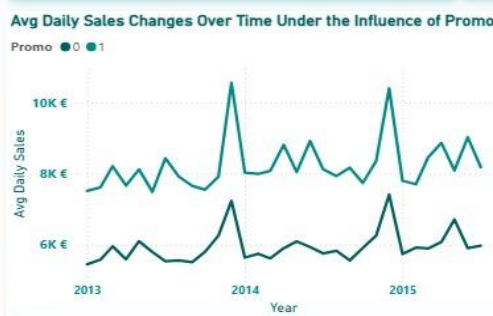
Store: Store Numbe

Date: 1/1/2013 7/31/2015

Store Type: All

State Holiday: All

Day Of Week: All



Open: 0 1

Promo2: 0 1

Promo: 0 1

Store: Store Numbe

Date: 1/1/2013 7/31/2015

Store Type: All

State Holiday: All

Day Of Week: All



11. UI/UX Design For ML Model Deployment

Rossmann store sales

Store ID

Prediction Date

Store Open?

Promo Active?

State Holiday

School Holiday?

Store Type

AI Sales Forecasting System

Predict future Rossmann sales using Machine Learning models.

Compare forecasting performance between:

- XGBoost
- Random Forest

Adjust store parameters from the sidebar and generate predictions instantly.

Adjust parameters from the sidebar then click Predict Sales.

School Holiday?

Store Type

Assortment

Competition Distance

Promo2 Active?

Select Models

Predict Sales

Prediction Results

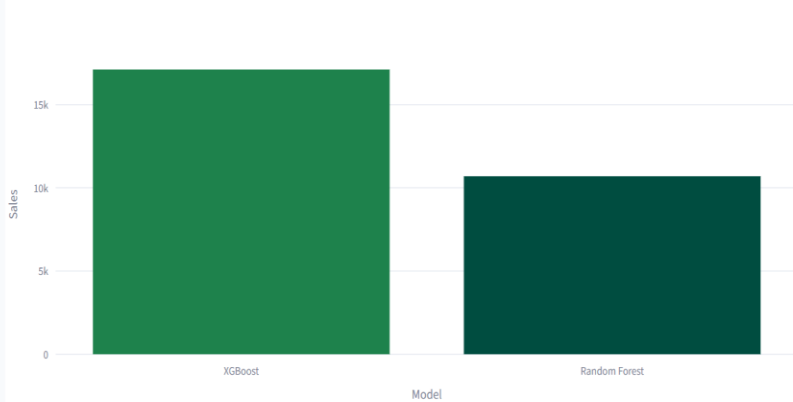
XGBoost Forecast

€ 17,112.09

Random Forest Forecast

€ 10,697.44

Models Comparison



Model	Sales
XGBoost	17,112.09
Random Forest	10,697.44

Implementation (Source Code & Execution)

12. Dataset Description

12.1. Overview

The dataset used in this project is sourced from the historical sales data of Rossmann drugstores. It contains time-series sales data for 1,115 distinct stores located across Germany. The primary objective is to forecast the daily Sales for up to six weeks in advance.

The data is split across multiple files, primarily capturing daily store operations, promotions, school/state holidays, and specific store characteristics (such as store type and competitor distance).

12.2. Dataset Files

The project utilizes the following main CSV files:

- train.csv: Contains historical data including sales, which is used to train the forecasting model.
- store.csv: Contains supplemental information about each specific store (e.g., store layout, competitor details, and promotion cycles).
- test.csv: Contains historical data excluding sales, used to evaluate the model's performance on future predictions.

12.3. Data Dictionary

Core Sales and Operational Features (From train.csv and test.csv)

- Store: An integer variable that represents a unique identification number for each individual Rossmann store.
- DayOfWeek: An integer variable representing the specific day of the week, coded from 1 for Monday to 7 for Sunday.
- Date: A date-formatted variable (YYYY-MM-DD) indicating the exact calendar date for the recorded daily entry.
- Sales: An integer variable representing the total monetary turnover or revenue generated by the store on that specific day. This serves as the primary Target Variable for our predictive model.
- Customers: An integer variable indicating the total number of customers who visited the store on that day.
- Open: A binary indicator where 1 means the store was open and 0 means the store was closed.

- **Promo:** A binary indicator specifying whether the store was actively running a commercial promotional campaign on that particular date (1 for active, 0 for inactive).
- **StateHoliday:** A categorical variable indicating if the date fell on a state holiday. It is encoded as follows: "a" for public holidays, "b" for Easter holidays, "c" for Christmas, and "0" for regular days with no holiday.
- **SchoolHoliday:** A binary indicator (1 or 0) showing whether the store's operations on that date were affected by the closure of public schools in its region.

Store Metadata and Competitor Features (From store.csv)

- **StoreType:** A categorical variable that differentiates between four distinct store models or layouts, labeled as "a", "b", "c", and "d".
- **Assortment:** A categorical variable describing the product assortment level offered by the store, encoded as "a" for basic, "b" for extra, and "c" for extended product ranges.
- **CompetitionDistance:** A float variable representing the straight-line distance in meters to the nearest competitor drugstore.
- **CompetitionOpenSinceMonth:** An integer variable indicating the calendar month (1 to 12) when the nearest competitor store originally opened.
- **CompetitionOpenSinceYear:** An integer variable providing the approximate calendar year when the nearest competitor store first became operational.
- **Promo2:** A binary indicator (1 or 0) that specifies whether the store participates in a continuing, consecutive promotional program.
- **Promo2SinceWeek:** An integer variable indicating the calendar week number of the year when the store first started its participation in the Promo2 campaign.
- **Promo2SinceYear:** An integer variable representing the calendar year when the store officially joined the Promo2 campaign.
- **PromoInterval:** A string and categorical feature that defines the specific consecutive months of the year when a new cycle of Promo2 is launched (for example, "Jan, Apr, Jul, Oct" or "Feb, May, Aug, Nov").

13. Data Cleaning & Preprocessing

13.1. Handling Missing Values

To ensure the quality of the dataset and optimize the performance of our forecasting models, a comprehensive data cleaning process was executed to handle missing fields within the store metadata dataset (store.csv). The missing values were processed based on the nature of each feature as follows:

- **Time-Based Competition and Promotional Features:** The features CompetitionOpenSinceMonth, CompetitionOpenSinceYear, Promo2SinceWeek, and Promo2SinceYear contained missing values because certain stores do not have nearby competitors or do not participate in the continuous promotion program (Promo2). These missing entries were strategically filled with 0 to represent the structural absence of an active competitor or promotion cycle.
- **Competition Distance:** The missing data points in the CompetitionDistance feature were imputed using the calculated Mean value of the available competition distances across the dataset. This approach maintains the overall statistical distribution of the feature without introducing significant bias or losing valuable rows of data.
- **Promotion Intervals:** For stores not enrolled in the consecutive promotional campaigns, the PromoInterval feature contained missing records. These null values were successfully replaced with the categorical label 'No Promo' to clearly distinguish these stores during downstream categorical encoding and modeling.

13.2. Data Type Transformation & Inconsistency Cleaning

Before integration, critical structural corrections were applied to the primary operational features in the training dataset to resolve data inconsistencies and format mismatches:

- **Datetime Conversion:** The Date column was originally loaded as a generic object (string) data type. It was explicitly converted into a proper datetime64 format. This transformation is a mandatory prerequisite for time-series parsing, alignment, and subsequent temporal feature extraction (such as extracting months, weeks, or seasons).
- **StateHoliday Inconsistency Resolution:** The StateHoliday feature suffered from mixed data type inconsistency, where non-holiday days were stored

simultaneously as an integer 0 and a string '0'. This artificial variation would cause standard categorical encoders to treat them as two distinct categories. To resolve this, all variations were standardized into a single string format ('0'), ensuring uniform representation of regular business days.

- **Dropping Closed Store Records (Data Filtering):** Historical rows where the store was closed ($\text{Open} == 0$) were explicitly dropped from the dataset. Since a closed store naturally generates zero sales, keeping these records introduces unnecessary noise and skews the model's learning process. Filtering them out allows the model to optimize its predictions strictly on active operational days.

13.3. Data Integration (Merging)

Following the imputation of missing values, the store metadata (`store_df`) was integrated with the historical training dataset (`train_df`) using a Left Join operation mapped on the unique Store identifier. This consolidation ensures that every daily sales log is enriched with its respective store profile, laying the groundwork for the feature engineering phase.

14. Feature Engineering & Categorical Encoding

14.1. Temporal & Calendar Feature Extraction

Given that sales forecasting is inherently a time-series problem, the raw Date feature was decomposed into several calendar components to capture seasonality and recurring trends:

- **Calendar Elements:** Year, Month, Day, and WeekofYear were extracted to help the model learn specific intra-month dynamics, annual trends, and weekly retail behaviors.
- **Seasonal Features:** Additional internal mapping of months was utilized to assist the algorithm in understanding broader seasonal variation and shifts in consumer spending habits throughout the fiscal year.

14.2. Operational Duration Metrics

To accurately represent the impact of market competition and marketing campaigns over time, two continuous duration metrics were calculated:

- **Competition Active Duration (`CompetitionOpen_month`):** Quantifies the total time (in months) that the nearest competitor has been operational relative to the transaction date. This allows the model to learn if a competitor's impact weakens over time as customer loyalty stabilizes.

- Promotion Active Duration (Promo2Open): Measures how long (in months) a store has been participating in the continuous promotional campaign (Promo2). This feature captures potential customer fatigue or compounding promotional success.

14.3. Categorical Variables Encoding

To convert non-numeric features into legal machine learning model inputs, distinct encoding strategies were deployed based on the nature of each variable:

1. Ordinal Encoding

Applied strictly to features that possess an inherent structural or sequential hierarchy:

- Assortment Mapping: The Assortment levels were converted into ordered numeric weights ('a' (Basic): 0, 'c' (Extended): 1, 'b' (Extra): 2) to respect the expanding variety and depth of the store's product ranges.

2. Nominal One-Hot Encoding

Applied to variables with no mathematical order or rank. Using scikit-learn's OneHotEncoder(sparse_output=False, handle_unknown='ignore'), the categories were expanded into independent binary columns:

- Targeted Columns: Encompassed StoreType (representing different store layouts) and StateHoliday (representing the type of holiday).
- Data Leakage Prevention: The encoder was strictly .fit() on the training partition and subsequently applied to transform both the training and evaluation/test datasets. The handle_unknown='ignore' parameter was configured to guarantee robust processing of unforeseen categories during live evaluation.

14.4. Feature Selection & Leaky Column Removal

Before finalizing the data matrices, a feature selection step was executed to drop redundant or invalid columns:

- PromoInterval: Dropped as its underlying seasonal data was already compressed and extracted through more structured promotional features.
- Customers (Data Leakage Removal): The Customers feature was explicitly removed from the training matrix. While highly correlated with Sales, future customer counts are strictly unknown during inference (and missing from test.csv). Removing it is mandatory to prevent severe Data Leakage and ensure realistic model evaluation.

15. Exploratory Data Analysis (EDA)

To extract actionable insights from the Rossmann dataset and validate our preprocessing strategy, an in-depth visual exploration was conducted across store operations, promotions, and seasonal factors.

15.1. Sales vs. Customers and Promo Efficiency

Analysis of the customer-to-sales distribution highlights a strong, positive linear correlation, establishing that customer foot traffic directly scales store revenue. Additionally, tracking the immediate impact of the standard promotional campaigns (Promo) reveals a substantial performance shift:

- When a promotion is active (Promo = 1), stores experience a +38.77% lift in overall Promo Impact, driving average daily sales up significantly.
- The data distribution clearly shifts upward and to the right during promotional periods, proving that Promo is one of the most critical predictors for our forecasting models.



15.2. Promo2 Performance and Efficiency Across Store Types

While short-term promotions (Promo) yield highly positive results, the continuous, long-term promotional program (Promo2) demonstrates a different behavior:

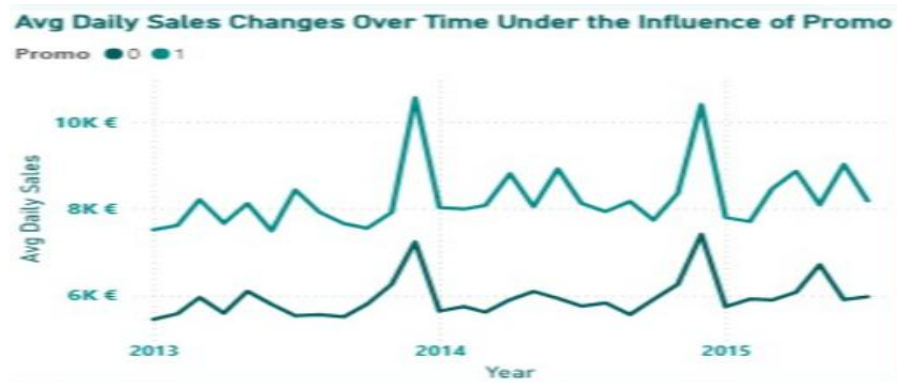
- Interestingly, the continuous campaign registers a -10.78% Promo2 Impact on average performance. This suggests potential customer fatigue or that stores enrolled in Promo2 are inherently struggling locations trying to capture market share.
- When analyzing this across different layouts (StoreType), Store Type 'b' stands out as the highest performer in average daily sales, whereas types 'a', 'c', and 'd' show steady operational sales but react differently when long-term campaigns are sustained.



15.3. Macroeconomic Trends and Multi-Year Seasonality

Evaluating the daily sales changes over time from 2013 through 2015 provides clear structural evidence of time-series seasonality:

- Annual Spikes: Consistent, massive revenue surges are visible at the end of each fiscal year (December holiday season spikes), followed by a predictable drop in early January.
- Promo vs. Non-Promo Timelines: Plotting the time-series curves with and without campaigns confirms that running a promotion successfully elevates the baseline sales floor throughout the entire year, maintaining a higher operational equilibrium compared to non-promotional periods.



16. Model Training and Evaluation

16.1. Data Splitting & Scaling Strategy

- **Chronological Time-Based Split:** Since sales data is time-dependent, random splitting cannot be used. Instead, a chronological split was applied using June 15, 2015 as the threshold. Data prior to this date was assigned for training, while the remaining subsequent data was reserved for testing. This successfully simulates a real-world scenario where the model forecasts future unseen sales.
- **Feature Scaling:** To normalize the data and speed up model convergence, Min-Max Scaling was applied using MinMaxScaler. Features and the target variable (Sales) were scaled down to a strict $[0, 1]$ range. After predictions were generated, an inverse transformation was applied to return the values to their original Euro currency scale for realistic assessment.

16.2. Evaluated Predictive Models

Three different machine learning and deep learning algorithms were trained and evaluated on the processed dataset:

LSTM (Long Short-Term Memory) - Deep Learning Approach

- Approach: A baseline Recurrent Neural Network (RNN) using the traditional LSTM architecture was tested to explore the potential of Deep Learning in capturing temporal sequences.
- Results: * Test R2 (Accuracy): ~20.00%
- Justification of Performance: The traditional LSTM struggled significantly with this dataset. This is because the Rossmann dataset is highly tabular and feature-rich (containing static metadata like store types, competitor distance, and promotional intervals) rather than being a pure, single-variable time series. Additionally,

- standard 2D data flattening fails to provide the 3D sequential formatting required for LSTMs to effectively map long-term dependencies, leading to suboptimal convergence.

Random Forest Regressor

- Approach: An ensemble of 100 decision trees was built to establish a strong tree-based bagging baseline. The model utilized parallel CPU cores (n_jobs=-1) to optimize training speed.
- Results: * Train R2 (Accuracy): 98.73%
 - Test R2 (Accuracy): 86.56%
 - Test Errors: MAE stood at 773.73€, with an RMSE of 1111.98€.

Extreme Gradient Boosting (XGBoost Regressor)

- Approach: To overcome the limitations of standard trees, a professional gradient boosting regressor was deployed with regularized boosting parameters (2,000 estimators, learning rate of 0.02, max depth of 10, and L1/L2 regularization) to maximize generalization.
- Results: * Train R2 (Accuracy): 95.56%
 - Test R2 (Accuracy): 90.46%
 - Test Errors: Dropped significantly to an MAE of 661.00€ and an RMSE of 936.89€.

Model Name	Train R2 (Accuracy)	Test R2 (Accuracy)	Test MAE	Test RMSE
LSTM (Deep Learning)	--	20.00%	High Deviation	High Deviation
Random Forest	98.73%	86.56%	773.73 €	1111.98 €
XGBoost (Winner)	95.56%	90.46%	661.00 €	936.89 €

16.3. Hyperparameter Tuning & Optimization

To further build upon our evaluated models and push forecasting accuracy to its maximum potential, a systematic hyperparameter optimization was executed using RandomizedSearchCV with K-Fold Cross-Validation. This process automatically searched multi-parameter configurations to discover the absolute optimal settings:

- Random Forest Optimization: The tuning process searched across various tree depths and leaf splits. The best configuration discovered was:
{'n_estimators': 100, 'max_depth': 30, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}.

This optimization stabilized the Cross-Validation score at 0.7984.

- XGBoost Optimization: The grid evaluated combinations of learning rates, tree configurations, and subsampling rules. The winning configuration was:

```
{'n_estimators': 1000, 'max_depth': 8, 'learning_rate': 0.1, 'subsample': 0.9,  
'objective': 'reg:squarederror'}
```

This optimization produced a dominant Cross-Validation R^2 score of 0.8612.

16.4. Key Insights & Final Model Choice

- Tabular vs. Sequential Models: The experiment proved that tree-based ensemble methods (Random Forest and XGBoost) are inherently superior for structured tabular data compared to standard Deep Learning architectures like LSTM, which require extensive sequencing and architectural tuning
- The Winner: XGBoost remains the designated winning model for this project, achieving a robust test accuracy of 90.46% and proving to be the most efficient and reliable architecture for Rossmann's forecasting needs.

17. MLOps and Experiment Tracking

17.1. Architecture Overview

To maintain enterprise-grade machine learning software standards, MLflow was integrated into the training and optimization pipeline as our core MLOps engine for model lifecycle management. This ensures that every computational experiment run by the team is fully auditable, structured, and reproducible.

17.2. Metrics & Parameter Logging

Rather than manually documenting empirical variants, the execution scripts were engineered to connect directly with an active MLflow backend (mlflow.db):

- Hyperparameter Logging: MLflow automatically captures the exact parameter combinations evaluated during the RandomizedSearchCV phase (including learning_rate, max_depth, subsample, and n_estimators).
- Performance Tracking: For every single trained iteration, the system logs evaluation metrics (R^2 accuracy, Mean Absolute Error, and Root Mean Squared Error) across both training and validation sets.
- Reproducibility: This rigorous logging mechanism eliminates code drift and provides complete historical visualization of our model development lifecycle, allowing the team to compare runs and justify the final selection scientifically.

17.3. Production Repository Structure

The project repository was engineered following a clean, modular Python packaging convention to enforce clean-code principles and separate production pipelines from testing environments:

- `src/`: Core package directory containing isolated operational scripts for data loading (`data_loader.py`), data cleansing (`preprocess.py`), and feature engineering (`feature_engineering.py`).
- `notebooks/`: Retained strictly for initial exploratory data analysis (EDA), data prototyping, and interactive visualization testing.
- `train.py` & `tune.py`: Standalone automation scripts dedicated to driving model training execution, logging to MLflow, and running automated hyperparameter grids.
- `main.py`: The central execution script that orchestrates the entire end-to-end pipeline, seamlessly connecting data loading, preprocessing, and model training in a single workflow.
- `predict_test.py`: A dedicated production inference script that loads the finalized trained models, executes batch predictions on the unseen test dataset, and automatically formats and exports the final submission.csv file required for evaluation.
- `st_app.py`: The production-ready application file that houses the final front-end user dashboard.

18. Model Deployment Using Streamlit

18.1. Production Architecture (Monolithic Engine)

The finalized optimized predictive models were deployed using Streamlit to form an integrated, single-instance AI Forecasting Dashboard tailored for operational business users and store managers. To minimize architecture overhead, the prediction pipeline is embedded directly within the application container runtime, processing user inputs and outputting real-time sales forecasts instantly.

18.2. In-Memory Model Caching

To achieve high responsiveness and sub-second inference times, the serialized winning models (`rossmann_xgboost_best.pkl`, `rossmann_rf_best.pkl`) and the target scalar (`scaler_y.pkl`) are loaded into the application layer using Streamlit's `@st.cache_resource` memory optimizer. This approach dynamically caches the heavy

model weights in memory, preventing redundant disk I/O operations whenever a user triggers a prediction.

18.3. User Interface and Dashboard Controls

The front-end user interface was fully customized using injected corporate CSS layouts styled around a premium deep green brand scheme (#004d40). Non-technical stakeholders can easily configure store conditions via intuitive interactive sidebar components:

- **Operational Variables:** Numerical and dropdown fields capture critical parameters like Store ID, opening status (Store Open?), and active promotional cycles (Promo Active? and Promo2 Active?).
- **Temporal and Environmental Variables:** Features a calendar date selector (Prediction Date), holiday types (State Holiday / School Holiday), and structural metrics like Store Type, Assortment, and Competition Distance.
- **Model Selection:** Includes a multiselect panel allowing users to dynamically choose between XGBoost and Random Forest to benchmark forecasting variations simultaneously.

18.4. Real-Time Inference Pipeline Flow

When an end-user inputs store parameters and triggers the "Predict Sales" action, the web application runs the following sequential production pipeline:

- **Feature Extraction:** The system automatically parses the selected date object to extract deep temporal dimensions (year, month, day, day_of_week, and week_of_year) and maps the corresponding calendar season.
- **Vector Construction:** Inputs are dynamically mapped into a strict 27-feature NumPy array vector that accurately mirrors the exact column shape expected by the trained estimators.
- **Model Prediction:** The input matrix is passed to the designated cached models (XGBoost and/or Random Forest) to execute the mathematical inference.
- **Inverse Transformation:** Because the target variables were normalized during training, the raw outputs are passed through `scaler_y.inverse_transform` to convert scaled fractions back into true Euro values.
- **UI Rendering:** The predicted sales values are displayed in bold KPI metrics. If multiple models are selected, a responsive Plotly Express bar chart is dynamically rendered to provide immediate performance comparison.

19. Project Limitations

While the developed forecasting system achieved high predictive accuracy and solid performance metrics, several inherent constraints were identified during the development lifecycle:

- **Historical and Outdated Dataset:** The training data spans from 2013 to mid-2015. In the retail and supply chain sectors, market dynamics, inflation rates, and consumer purchasing behaviors shift drastically over time. Relying on decade-old data limits the model's ability to accurately capture modern retail trends, meaning the dataset requires a substantial update with recent sales figures before any actual market deployment.
- **Hardware & Compute Constraints:** Training complex ensemble algorithms like Random Forest and XGBoost across millions of rows required significant CPU compute time. Due to limited local hardware resources, the hyperparameter tuning phase using RandomizedSearchCV had to be constrained to a selective number of iterations and folds ($n_iter=15$) rather than an exhaustive grid search.
- **Lack of Unstructured/External Data:** The models rely entirely on the structured historical parameters provided in the Rossmann dataset. In real-world retail, sales are heavily influenced by unstructured external factors such as local weather conditions, live supply chain disruptions, competitors' real-time pricing changes, and macroeconomic shifts, which were unavailable in our data.
- **Monolithic Architecture:** The current production app couples the front-end interface and the machine learning inference pipeline within a single Monolithic Streamlit deployment. While highly efficient for prototype testing, this architecture does not support independent scaling of the model weights from the user interface under heavy corporate user traffic.

20. Future Improvements

To scale this project into a robust, enterprise-grade production platform, the following tactical and technical enhancements are proposed:

- **Dataset Refresh & Active Pipeline Integration:** Update the training pipeline with contemporary retail transactional data and re-baseline the models to align with modern economic standards and current shopping behaviors.

- Continuous Integration & Automated Retraining (CI/CD): Deploy an automated MLOps pipeline using GitHub Actions combined with MLflow Registry. This pipeline will automatically monitor the live model for data drift and trigger periodic retraining schedules whenever prediction errors exceed a predefined threshold.
- Deep Learning Sequence Optimization: Redesign the sequential deep learning pipeline by formatting the historical tabular data into true 3D temporal sliding windows. This will allow advanced architectures like Temporal Convolutional Networks (TCNs) or optimized LSTMs to properly capture long-term sequential behavior and benchmark them against the winning tree-based models.
- Transition to Microservices Architecture (FastAPI): Separate the machine learning inference pipeline into an independent, containerized RESTful API using FastAPI or Flask. This will allow the Streamlit dashboard to act as a lightweight client making asynchronous HTTP requests, optimizing resource allocation and scalability.
- Feature Enrichment via API Integration: Integrate external open-source APIs to dynamically inject live data streams into the preprocessing pipeline, such as historical weather data and local consumer sentiment analysis, to capture hidden variations in retail shopping behavior